CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 08:
Mid-Semester Review

Course taught by: Zubair Irshad

Midterm Exam Logistics

- Date: Monday 20th October 2025, 5.40-7.00pm
- Part (1) Multiple Choice Questions - this part will be of 35 minutes duration (**5.40-6.15pm**)
- Part (2) Programming - this part will be of 35 minutes duration. (**6.25-7.00pm**)
- **Syllabus:** Introductory C++, Pointers, Arrays (1-D and 2-D), Functions, Classes and Objects, Operator Overloading, Rule of 3.

Topic 1 Introduction — What We Covered

- Object-Oriented Programming (OOP) vs Procedural Programming
- Why OOP matters → reusability, modularity, maintainability
- Why C++ → performance + control (vs Python)
- Structure of a C++ program → #include, main(), cout, cin
- Variables & Data Types → int, char, float, double, bool
- Type conversions & casting (static_cast, C-style cast)
- Control flow → if-else, loops (for, while, do-while)
- Functions → definition, return type, parameters

Anatomy of a C++ Program

```
#include
<iostream>
using namespace
std;


int main() {
    cout << "Hello
World!" << endl;
    return 0;
}
```

Key parts:

- `#include` → preprocessor directive
- `using namespace std;` → simplifies access to `cout`, `cin`
- `main()` → entry point; `int` return type
- `cout` / `cin` → stream objects for I/O
- `<<` / `>>` → insertion & extraction operators

Data Types and Conversions

Type	Example	Bytes	Note
int	42	4	whole numbers
float	3.14 f	4	~6 decimal digits
double	3.14159	8	~15 digits
char	'A'	1	ASCII character
bool	true	1	boolean values

Implicit Casting (Type Promotion)

- When two different data types are used in an expression, the *lower-ranked* type is automatically promoted to the *higher-ranked* type.
- `long double > double > float > long > int > short > char`

Explicit Casting (Type Forcing)

- We can manually control conversion using **C-style** or **C++ style** casting.

```
int a = 7, b = 2;
double c = static_cast<double>(a) / b;
cout << c;   // 3.5
```

Operators & Control Flow

- Arithmetic → +, -, *, /, %
- Relational → ==, !=, >, <, >=, <=
- Logical → &&, ||, !

- **Loop Example**

```
for(int i=0; i<5; i++)
    cout << i << " ";
```

- **Decision Example**

```
if (x > 0)
   cout << "Positive";
else
    cout << "Non-positive";
```

Practice Questions

- **Q1.** Which statement about main() in C++ is **true**?

  A. It must return void.
  B. It is the entry point for all C++ programs.
  C. It can appear multiple times.
  D. It cannot return a value.

Practice Questions

- **Q1.** Which statement about main() in C++ is **true**?

  A. It must return void.
  B. It is the entry point for all C++ programs.
  C. It can appear multiple times.
  D. It cannot return a value.

Practice Questions

- **Q2.** What is the result?

```
int a = 7 % 3 + 4 / 2;
cout << a;
```

Practice Questions

- **Q2.** What is the result?

```
int a = 7 % 3 + 4 / 2;
cout << a;
```

A. 1 + 2 = 3 → prints 3

Practice Questions

- **Q3.** What is the output?

```
int a = 5, b = 2, c = 3;
cout << (a > b) + (b > c);
```

A. 0 B. 1 C. 2 D. True

Practice Questions

- **Q3.** What is the output?

```
int a = 5, b = 2, c = 3;
cout << (a > b) + (b > c);
```

A. 0 **B. 1** C. 2 D. True

Explanation: (a>b) → true (1), (b>c) → false (0) → 1+0=1.

Topic 2: Arrays — Overview

- An **array** is a *collection of elements of the same type* stored in contiguous memory (They are ***homogeneous* and *fixed-size***)
- `int list[4] = {1, 2, 3, 4};`
- **Indexing:** starts from 0 → `size – 1`
- **Access:** list[2] = 10;
- **Types of arrays:**
  - **1-D:** linear storage (int a[5];)
  - **2-D:** table-like (int m[3][4];)
    - Int m[3][] would be an invalid declaration, why?
- No automatic bound-checking → programmer's responsibility.

Arrays -- Traversing

- Loops are essential for traversal.
- **1-D array traversal:**

```
for(int i=0; i<4; i++)
    cout << a[i] << " ";
```

- **2-D Array Example**

```
int m[2][3] = {{1,2,3},{4,5,6}};
for(int i=0;i<2;i++)
    for(int j=0;j<3;j++)
        cout << m[i][j] << " ";
```

- → Output 1 2 3 4 5 6
- Think of a 2-D array as *array of arrays.*

Arrays & Functions

- Arrays can be passed to functions *by reference (implicitly)*
- `void print(int arr[], int size);`
- The **size** of the first dimension is optional in a parameter; others must be known:
- `void show(double data[][12], int row);`
- Always pass array length as a separate parameter or constant i.e. n can be array length

Strings (Character Arrays) and Common Pitfalls

- C-Strings = 1-D arrays of char ending with a **null character \0**
- char name[10] = "Ali"; Always allocate space for the terminating \0.

Common Pitfalls to avoid:

- Accessing `a[size]` → **out-of-bounds** (undefined behavior)
- Forgetting the null terminator in C-strings. Is automatically added when accepting using `cin`.
- Passing 2-D arrays without fixing column size
- Mixing indices (`arr[i][j]` vs `arr[j][i]`)
- Not initializing arrays before use

Practice Questions

- **Q1.** What is printed?

```
int a[2][2] = {{1,2},{3,4}};
for(int i=0;i<2;i++)
    for(int j=0;j<2;j++)
        cout << a[j][i] << " ";
```

A. 1 3 2 4 B. 1 2 3 4 C. 4 3 2 1 D. 2 1 4 3

Practice Questions

- **Q1.** What is printed?

```
int a[2][2] = {{1,2},{3,4}};
for(int i=0;i<2;i++)
    for(int j=0;j<2;j++)
        cout << a[j][i] << " ";
```

A. **1 3 2 4** B. 1 2 3 4 C. 4 3 2 1 D. 2 1 4 3

Practice Questions

- **Remove an Element from an Array**
- Write a C++ program that removes an element from an integer array at a specified position and then displays the new array.

- Input/Outputs:
- Enter number of elements: 5
- Enter elements: 10 20 30 40 50
- Enter position to remove: 2
- New array: 10 20 40 50

Practice Questions

Potential Solution given `pos` and `n` i.e. Number of elments and array `arr`

```
for (int i = pos; i < n - 1; i++){
    arr[i] = arr[i + 1];
{
```

Topic 3 Pointers — Introduction

- A **pointer** is a variable that stores the *memory address* of another variable.
- int x = 43;
- int* p = &x; // p stores the address of x
- cout << *p; // dereference → prints 43
- & → **address-of** operator
- * → **dereference** operator
- **Pointer type must match** the type of variable it points to (int*, char*, double*, etc.)

Pass by Pointer & Reference & Value

- **Pass by Value** → function gets a copy (original unchanged)
- **Pass by Pointer** → function gets the *address*, can modify original
- `void increment(int* p){(*p)++; }`
- `int a = 5;`
- `increment(&a); // a = 6`
- **Pass by Reference** → alias (cleaner syntax)
- `void increment(int& r){ r++; }`
- `int a = 5;`
- `increment(a);  // a = 6`

Pointers and Arrays

- The **name of an array** acts as a pointer to its first element.
- `int a[] = {5, 3, 4, 2, 1};`
- `cout << *a;  // 5`
- `cout << *(a+3);  // 2`
- `a[i] ≡ *(a + i)`
- Pointers allow arithmetic operations (+, -, comparison).
- Be careful not to dereference out-of-bounds addresses.

Dynamic Memory Allocation

- Use **new** to allocate memory at runtime (heap).
- `int* arr = new int[n];`
- Use **delete** or **delete[]** to free memory.
- `delete[] arr;`
- Also set `arr = nullptr;  // prevents accidental reuse`
- **Static array** → stack, auto-deleted.
- **Dynamic array** → heap, must be explicitly deleted.

Double Pointers

- A **pointer to a pointer** (int**) stores the address of another pointer.
- `int i = 10, j = 25;`
- `int* p1 = &i;`
- `int* p2 = &j;`
- `int** pp = &p2;`
- `cout << **pp; // prints 25`
- Common use: dynamic 2D arrays (`int** arr = new int*[rows];`)
  - Each row pointer allocated separately.
  - Always deallocate using nested delete[].
- **Important:** Practice 2D string pointer examples from pointer slide 46-48 with various indexing

Practice Questions

- **Q1.** What is printed?

```
int arr[2][3] = {{1,2,3},{4,5,6}};
cout << *(*(arr + 1) + 2);
```

A. 3 B. 4 C. 6 D. Compilation error

Practice Questions

- **Q2.** What is printed?

```
int arr[2][3] = {{1,2,3},{4,5,6}};
cout << *(*(arr + 1) + 2);
```

A. 3 B. 4 **C. 6** D. Compilation error

(arr + 1) points to the address of the 2nd row {4,5,6},
Adding +2 leads us to → 6.
Remember we have to dereference with pointer notation to get the value
i.e. a[i] = *(a+i)

Practice Questions

- **Q2.** What will the following print?

```
int a = 3;
int* p = &a;
int& r = a;
r = *p + 2;
cout << a;
```

A. 2 B. 3 C. 5 D. 6

Practice Questions

- **Q2.** What will the following print?

```
int a = 3;
int* p = &a;
int& r = a;
r = *p + 2;
cout << a;
```

A. 2 B. 3 **C. 5** D. 6

both *p and r refer to same variable → a = 5.

Practice Questions

- **Q3.** Which statement is **true**?

  A. A pointer must be initialized when declared.
  B. A reference must be initialized when declared.
  C. A reference can point to multiple variables at runtime.
  D. A pointer cannot hold NULL.

Practice Questions

- **Q3.** Which statement is **true**?

  A. A pointer must be initialized when declared.
  B. **A reference must be initialized when declared.**
  C. A reference can point to multiple variables at runtime.
  D. A pointer cannot hold NULL.

- *Explanation:* References are aliases; must bind immediately to a valid variable.

Function Overloading Essentials

- Multiple functions with the **same name** but **different signatures** (parameter type, number, or order).
- `int add(int a, int b);`
- `double add(double a, double b);`
- `int add(int a, int b, int c);`

- **What is Allowed?** differing parameter types, count, or order
- **What is not allowed?** differing only in return type

Topic 5 Classes & Objects: Introduction

- A **class** is a **blueprint** for creating objects. It combines **data members (attributes)** and **member functions (methods)**.

```
class Car {
    string brand;
    int year;
public:
    void show() {cout << brand << " " << year;}
};
```

- An **object** is an **instance** of a class.
- Car c1; // object of class Car
- **Access Specifiers:**
  - private → accessible only within class
  - public → accessible anywhere

Notion of **this**

- In C++, **this** is a special pointer that exists inside every non-static member function of a class.
- It points to **the current object —** the one that called the function.
- Example:

```
class Student {
public:
    int age;
    void setAge(int age) {
        this->age = age;  // "this->age" refers to the object's member,
"age" is the parameter
    }
};
```

Key Takeaway: this helps the object refer to itself i.e. Student s1 so **this** refers to **s1**

Constructors & Initialization

- **Constructor:** special member function with **same name as class**. Automatically called when an object is created.

```
class Box {
    int l, w;
public:
    Box() { l = 0; w = 0; }           // default constructor
    Box(int x, int y) : l(x), w(y) {} // parameterized
(initializer list)
};
```

- No return type (not even void).
- Can be **overloaded** (different parameters).
- **Initializer List** preferred for const or reference members.

Copy Constructor & Destructors

- **Copy constructor** creates a new object as a copy of an existing one.

```
Box b1(3, 4);
Box b2 = b1;    // invokes copy constructor
```

- **Default copy constructor:** shallow copy (copies addresses).
- **Custom copy constructor:** deep copy (allocates new memory).

```
Box(const Box& obj) {
    l = new int(*obj.l);   // allocate new memory and copy
value
    }
```

- Key Takeaway: If your class has **pointers**, default shallow copy can cause
  - **Double deletion**
  - **Unintended shared memory**
- **Destructor**: special member function with same name as class, preceded by ~

```
~Box() { delete[] arr;}
```

Objects as Arguments & Returns

- Class objects can be **passed to** and **returned from** functions.

```
class Rectangle {
    int w, h;
public:
    Rectangle add(Rectangle r) {
        Rectangle temp;
        temp.w = w + r.w;
        temp.h = h + r.h;
        return temp;
    }
};
```

- Functions can access **private data** of other objects of the same class.
- Use const & to avoid unnecessary copies in parameters.

Practice Questions

- **Q1.** What is printed?

```
class Box {
public:
    int l;
    Box(int a) { l = a; }
};
int main() {
    Box b1(10);
    Box b2 = b1;
    b2.l = 20;
    cout << b1.l;
}
```

A. 20 B. 10 C. 0 D. Garbage

Practice Questions

- **Q1.** What is printed?

```
class Box {
public:
    int l;
    Box(int a) { l = a; }
};
int main() {
    Box b1(10);
    Box b2 = b1;
    b2.l = 20;
    cout << b1.l;
}
```

A. 20 **B. 10** C. 0 D. Garbage (Default copy constructor performs member-wise copy, custom not needed!

Practice Questions

- **Q3.** What will be printed?

```
class Test {
public:
    Test() { cout << "C"; }
    ~Test() { cout << "D"; }
};

int main() {
    Test t1, t2;
}
```

- A. CCDD B. DDCC C. CCDD D. CDDC.

Practice Questions

- **Q3.** What will be printed?

```
class Test {
public:
    Test() { cout << "C"; }
    ~Test() { cout << "D"; }
};

int main() {
    Test t1, t2;
}
```

A. **CCDD** B. DDCC C. CCDD D. CDDC

Explanation: Constructors called in order → C C, destructors in reverse order → D D.

Practice Questions

```
class Data {
public:
    int* p;
    Data(int val) { p = new int(val); }
    ~Data() { delete p; }
};
int main() {
    Data d1(10);
    Data d2 = d1;
}
```

A. Works fine B. Causes double deletion C. Memory leak D. Compilation error

Practise Questions

```
class Data {
public:
    int* p;
    Data(int val) { p = new int(val); }
    ~Data() { delete p; }
};
int main() {
    Data d1(10);
    Data d2 = d1;
}
```

A. Works fine **B. Causes double deletion** C. Memory leak D. Compilation error

Explanation: default copy constructor copies pointer → both destructors delete same memory → crash.

Topic 6 Operator Overloading

- Allows user-defined types to behave like built-in types.
- Achieved by defining a special function with the operator keyword.

```
class Distance {
    int meters;
public:
    Distance(int m=0): meters(m) {}
    Distance operator+(Distance d) {return Distance(meters +
d.meters);}
};
```

- **Rules:**
  - At least one operand must be a user-defined type.
  - Precedence & associativity **cannot** be changed.
  - You **cannot** create new operators (** invalid).
- Types:
  - **Binary operators:** +, -, *, /, ==, etc.

Topic 7 Rule of 3

- **Rule of 3:**
  If a class defines **any one** of the following, it should define **all three**:
  - **Destructor** → cleans up resources
  - **Copy Constructor** → creates a copy of an existing object
  - **Copy Assignment Operator** → assigns one existing object to another
- **Why?** To prevent:
  - **Shallow copies** (shared pointers)
  - **Double deletions**
  - **Memory leaks**

Rule of 3 followed: An example

```
class Array {
    int* data;
public:
    Array(int val){ data = new int(val); }
    ~Array(){ delete data; } // Destructor
    Array(const Array& a){ data = new int(*a.data); } // Custom copy constructor
    Array& operator=(const Array& a){  // Copy Assignment Operator
        if(this != &a){
            delete data;
            data = new int(*a.data);
        }
        return *this;
    }
};
```

Practice Question

- **Q1.** Which operator is explicitly defined when following Rule of 3?

  A. operator+
  B. operator=
  C. operator<<
  D. operator()

Practice Question

- **Q1.** Which operator is explicitly defined when following Rule of 3?

  A. operator+
  B. operator=
  C. operator<<
  D. operator()

Practice Questions

- **Q2.** What will this print?

```
class Box {
    int size;
public:
    Box(int s=0): size(s) {}
    bool operator==(Box b) { return size == b.size; }
};
int main() {
    Box b1(10), b2(10), b3(5);
    cout << (b1 == b2) << " " << (b1 == b3);
}
```

A. 1 0 B. 0 1 C. 1 1 D. 0 0

Practice Questions

- **Q3.** What will this print?

```
class Box {
    int size;
public:
    Box(int s=0): size(s) {}
    bool operator==(Box b) { return size == b.size; }
};
int main() {
    Box b1(10), b2(10), b3(5);
    cout << (b1 == b2) << " " << (b1 == b3);
}
```

A. **1 0** B. 0 1 C. 1 1 D. 0 0

Explanation: b1 == b2 → true → 1, b1 == b3 → false → 0.

Practice Questions

- **Q3.** What will this print?

```
class Box {
    int size;
public:
    Box(int s=0): size(s) {}
    bool operator==(Box b) { return size == b.size; }
};
int main() {
    Box b1(10), b2(10), b3(5);
    cout << (b1 == b2) << " " << (b1 == b3);
}
```

A. **1 0** B. 0 1 C. 1 1 D. 0 0

Explanation: b1 == b2 → true → 1, b1 == b3 → false → 0.

Good Luck preparing for Midterm!

- For coding practice, just revise each topic's respective lab's coding component and make sure you know your C++ basics well!